

Controlador de motor DC

Bernal Facundo, Faraco Nahuel

Técnicas Digitales III, UTN FRA

Avellaneda, Argentina

facundober03@gmail.com

nahueelfaracco@gmail.com

Abstract— El proyecto consiste en el desarrollo de un controlador para motores DC de baja potencia. Se realiza un control de la posición angular del eje mediante una técnica PID. Se basa en un microcontrolador ESP32-S3, ejecutando un FreeRTOS. El controlador se comunica mediante UART. Como encoder de posición se utiliza un AS5600.

Palabras clave— Motor DC – FreeRTOS – ESP32-S3 – Control PID – UART – AS5600.

ÍNDICE

I. FUNDAMENTACIÓN.....	1
A. Problemática.....	1
B. Solución.....	1
C. Fundamentación Software.....	1
D. Fundamentación Hardware.....	1
II. OBJETIVOS.....	2
III. ALCANCE.....	2
IV. DIAGRAMA DE BLOQUES.....	2
V. HARDWARE.....	2
A. ESP32-S3.....	2
B. Encoder magnético.....	2
C. Puente H.....	3
D. Motor DC.....	3
E. Display LCD2404.....	3
F. Sensor de corriente.....	3
VI. ESQUEMÁTICO y PCB.....	5
VII. SOFTWARE.....	6
VIII. CONCLUSIONES.....	6
IX. REFERENCIAS.....	6
X. ANEXOS.....	7

I. FUNDAMENTACIÓN

A. Problemática

En muchos procesos donde se utilizan motores, es necesario controlar con precisión la posición angular del eje. Esta posición debe ser independiente de la carga aplicada al eje, y se debe mantener frente a variaciones del torque en el eje.

Además, se precisa una implementación de técnicas de arranque suave (arranque en escalones, o rampas, entre otros). Estos parámetros de arranque deben poder ser controlados para adaptarse a las condiciones de funcionamiento del motor.

B. Solución

Frente a esta problemática, se propone un control PID (Proporcional Integral Derivativo), que establece una posición angular determinada del eje (configurada por el usuario). Esta posición es medida mediante un encoder magnético de posición, el AS5600 (de 12 bits o 4096 pasos por vuelta). De esta forma, se logra un control preciso de la posición del eje, independiente de la

carga.

El controlador tiene la capacidad de detectar obstáculos en el giro, y llegar a la misma posición en sentido contrario.

Se implementa una rutina de arranque suave, con parámetros configurables por el usuario. Así se logran arranques directos, arranques por pasos o arranques por rampa.

C. Fundamentación Software

Como base del proyecto se utilizó un sistema operativo de tiempo real: el FreeRTOS. Permite la organización del software en distintas tareas fácilmente identificables. De esta forma se logra mayor facilidad al solucionar problemas en el funcionamiento. Las tareas se organizan mediante niveles de prioridad, lo que facilita la ejecución correcta del programa.

El flujo de datos entre tareas, y su tiempo de ejecución, se organizan mediante semáforos y colas.

Para controlar los distintos periféricos se utilizan las denominadas “tareas guardianas”, que se encargan de hacer de interfaz entre las tareas que requieren su utilización y el periférico como tal.

Para la interfaz máquina-humano (HMI), se utilizan comandos a través de una PC mediante UART, o una serie de pulsadores en el propio equipo. Los comandos mediante los pulsadores se controlan mediante interrupciones por GPIO y semáforos.

Los datos más importantes del funcionamiento son mostrados en un display LCD de 20x4 caracteres. El display LCD se comunica con el ESP32 mediante un protocolo I2C.

La posición angular del eje (la salida del AS5600) se obtiene mediante el protocolo I2C.

La detección de obstáculos se hará mediante una resistencia de bajo valor en serie con la alimentación del motor, cuyo valor será leído por el ADC del microcontrolador. Al detectar un valor mayor a determinado umbral, se supone un obstáculo en el giro.

Para que los parámetros configurados no se pierdan al apagar el equipo, se almacena el último estado del controlador en la memoria flash interna del ESP32-S3. Se almacenan parámetros PID, de arranque, y umbrales de corriente.

D. Fundamentación Hardware

Para la implementación del controlador se utiliza como base al microcontrolador ESP32-S3. El elemento más crítico del sistema es el encoder. Se utiliza un encoder magnético de posición, que permite una lectura suficientemente precisa del ángulo del eje (con una resolución de 0.09°). Para demostrar su funcionamiento, se

utiliza un motor de baja potencia, de 24V, alimentado mediante un puente H basado en un L298N. Sin embargo, es posible escalar a mayores potencias sin dificultad.

Los dispositivos de HMI mediante UART permiten un control en tiempo real de los parámetros del motor (ángulo, velocidad, corriente, etc.), además de los parámetros del controlador PID (K_p , K_i , K_d). Se pueden enviar comandos de control a través de la comunicación UART.

Para comandos de arranque y parada (entre otros) se puede además utilizar una botonera en el propio equipo. Los datos importantes del funcionamiento del sistema son mostrados (además de la pantalla en el dispositivo UART) en un LCD de 20x4 caracteres.

II. OBJETIVOS

Los principales objetivos para el proyecto son los siguientes:

- Controlar mediante técnicas PID un motor de DC.
- Implementar técnicas de arranque suave en el controlador PID.
- Mostrar parámetros de funcionamiento del motor DC.
- Mostrar y permitir la calibración de parámetros de funcionamiento del controlador PID y del arranque suave.
- Detectar obstáculos en el giro del eje del motor.
- Controlar parámetros mediante una PC, comunicada mediante UART.

III. ALCANCE

Se pretende que el controlador tenga las siguientes características básicas:

- Resolución de la posición angular del eje de 1°.
- Control de motores DC de al menos 24VDC y 2A.
- Control PID.
- Arranque suave.
- Control mediante UART.

IV. DIAGRAMA DE BLOQUES

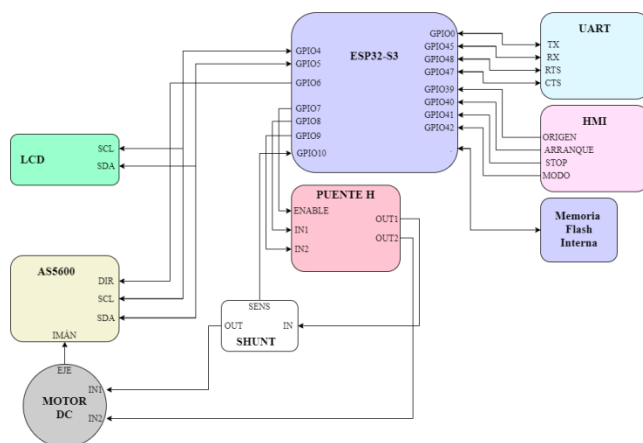


Fig. 1 Diagrama de bloques del controlador.

V. HARDWARE

A. ESP32-S3

El núcleo del proyecto se desarrolla sobre el ESP32-S3. El microcontrolador tiene la capacidad de ejecutar FreeRTOS, y posee varios periféricos que se aprovechan en la aplicación.

Por ejemplo, la placa de desarrollo junto con el microcontrolador posee una memoria Flash de 4MB, varios conversores AD, y comunicación I2C y UART.

El ESP32-S3 es el encargado de generar los pulsos para el control del motor, y de realizar el procesamiento del control PID.

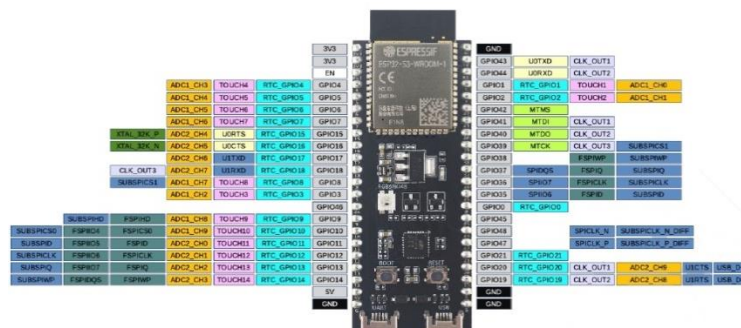


Fig. 2 Pinout de ESP32-S3.

El ESP32-S3 posee un procesador de doble núcleo XTensa LX7, con una frecuencia de Clock de hasta 240 MHz.

B. Encoder magnético

El encoder utilizado para obtener la posición angular del eje es el AS5600. Es un encoder absoluto (0° a 360°), que detecta la posición angular de un campo magnético. El campo magnético es generado por un imán permanente adosado directamente al eje del motor.

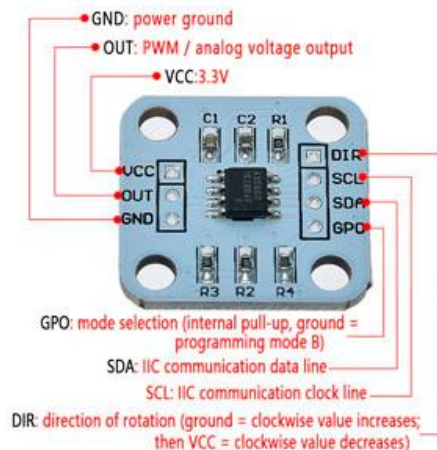


Fig. 3 Pinout del encoder magnético.

El AS5600 tiene una resolución de aproximadamente 0.1° , suficiente para la aplicación a desarrollar. Tiene una salida analógica (utilizada para control de velocidad angular del eje), pero se utilizará la salida en I2C. Tiene una longitud de palabra de 12bits.

C. Puente H

Para la etapa de potencia, y teniendo en cuenta el motor utilizado para el proyecto, se utiliza un puente H basado en el integrado L298N. Tiene la posibilidad de ser alimentado con una tensión de continua de 0V a 36V, y su corriente de salida continua máxima es de 2A.

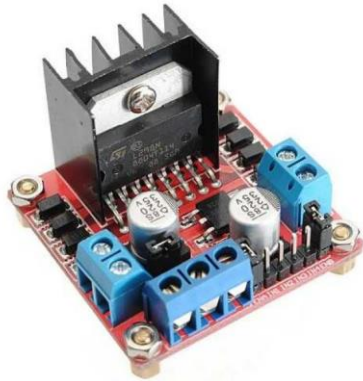


Fig. 4 Puente H.

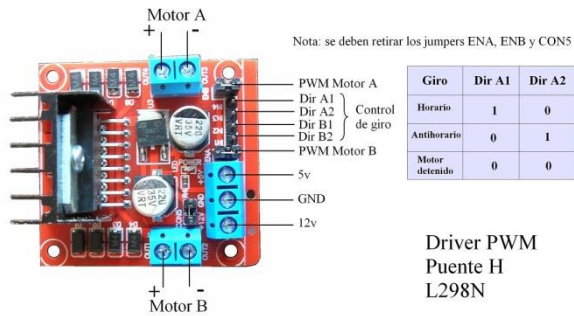


Fig. 5 Pinout del puente H.

El puente H toma los datos enviados por el microcontrolador del sentido de giro del motor, y los pulsos de PWM necesarios para el control de la posición.

Para escalar la potencia del controlador, el puente H es la única limitante del sistema. Para controlar motores de mayor potencia, solo sería necesario elegir un puente H adecuado.

D. Motor DC

Para la demostración del sistema de control se utiliza un motor de baja potencia. Es un motor de 24VDC de alimentación nominal, con escobillas.



Fig. 6 Motor DC utilizado.

En el eje se adosa el imán que permite al encoder detectar la posición angular. El posicionamiento del imán respecto al encoder es fundamental para el correcto funcionamiento del sistema.

E. Display LCD

Para mostrar los datos más importantes del funcionamiento del sistema, se utiliza un display LCD de 20x4 caracteres, controlado mediante un adaptador I2C basado en el PCF8574.



Fig. 7 Display LCD 24x4 caracteres.



Fig. 8 Adaptador I2C PCF8574.

F. Sensor de corriente

Como sensor de corriente se utiliza simplemente una resistencia de bajo valor en serie con la alimentación del motor.



Fig. 9 Resistencias Shunt.

El valor de la caída de tensión (proporcional a la corriente del motor) es leído por el ADC del microcontrolador y, pasado cierto umbral configurable, se detecta un bloqueo del eje. De esta forma, se busca llegar a la misma posición pero en sentido de giro contrario.

VI. ESQUEMÁTICO Y PCB

VII. SOFTWARE

El programa está organizado en varias tareas, con funciones claramente distinguibles. Estas tareas son:

- **Función MAIN:** Es la función que crea todas las demás tareas, crea las colas y semáforos e inicializa los protocolos UART e I2C. Crea todo lo necesario para el funcionamiento del sistema operativo de tiempo real. Su función culmina con la puesta en marcha del Scheduler de FreeRTOS.
- **Task_PID:** La tarea principal. Es la encargada de la lectura de la posición del eje y el valor de la posición deseada. Además, debe manejar el control según los parámetros del PID y del sentido de giro. Por lo tanto, su función es generar la información necesaria para el control del puente H.
- **Task_H_controller:** Es la tarea guardiana del puente H. Recibe la información necesaria para su funcionamiento, y genera los pulsos y valores de tensión para el control del motor DC.
- **Task_current_sense:** Es la tarea encargada del sensado de corriente. Detecta los obstáculos en el giro del eje. Su salida es una señal de obstáculo.
- **Task_flash:** Es la tarea encargada de guardar en la memoria flash determinados parámetros de funcionamiento, que no se deben perder al apagar el equipo. Realiza el guardado cada determinado tiempo.
- **Task_UART_guard:** Es la tarea guardiana del protocolo UART. Recibe parámetros que deben ser mostrados en pantalla, y envía comandos que fueron ingresados por el usuario.
- **Task_encoder_controller:** Es la tarea encargada de la comunicación con el encoder. Recibe los datos de posición y sentido de giro del motor, y los transforma en información útil para el control PID.

- **Task_LCD_controller:** Es la tarea encargada de la comunicación con el LCD. Recibe los datos que deben ser mostrados en el LCD.
- **Task_I2C_guard:** Es la tarea guardiana del protocolo I2C. Recibe los datos que deben ser enviados al LCD y los datos recibidos por el encoder, y logra establecer la comunicación.
- **Task_HMI_controller:** Es la tarea que resuelve las interrupciones generadas por entradas del usuario en la botonera propia del equipo. Se maneja mediante GPIO, y proporciona información a todas las demás tareas.

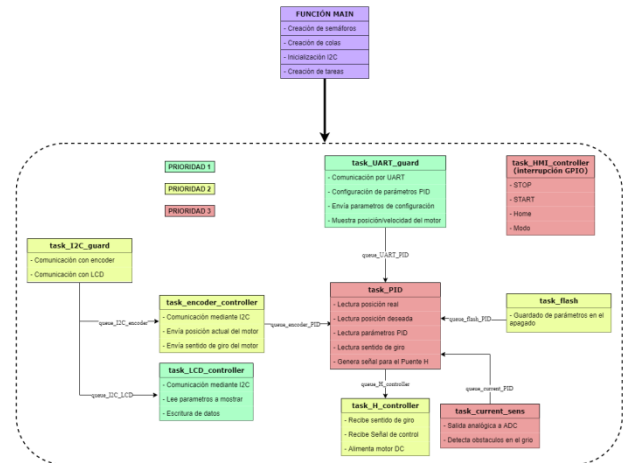


Fig. 13 Diagrama de tareas del programa.

X. ANEXOS

Anexo 1. Diagrama en bloques

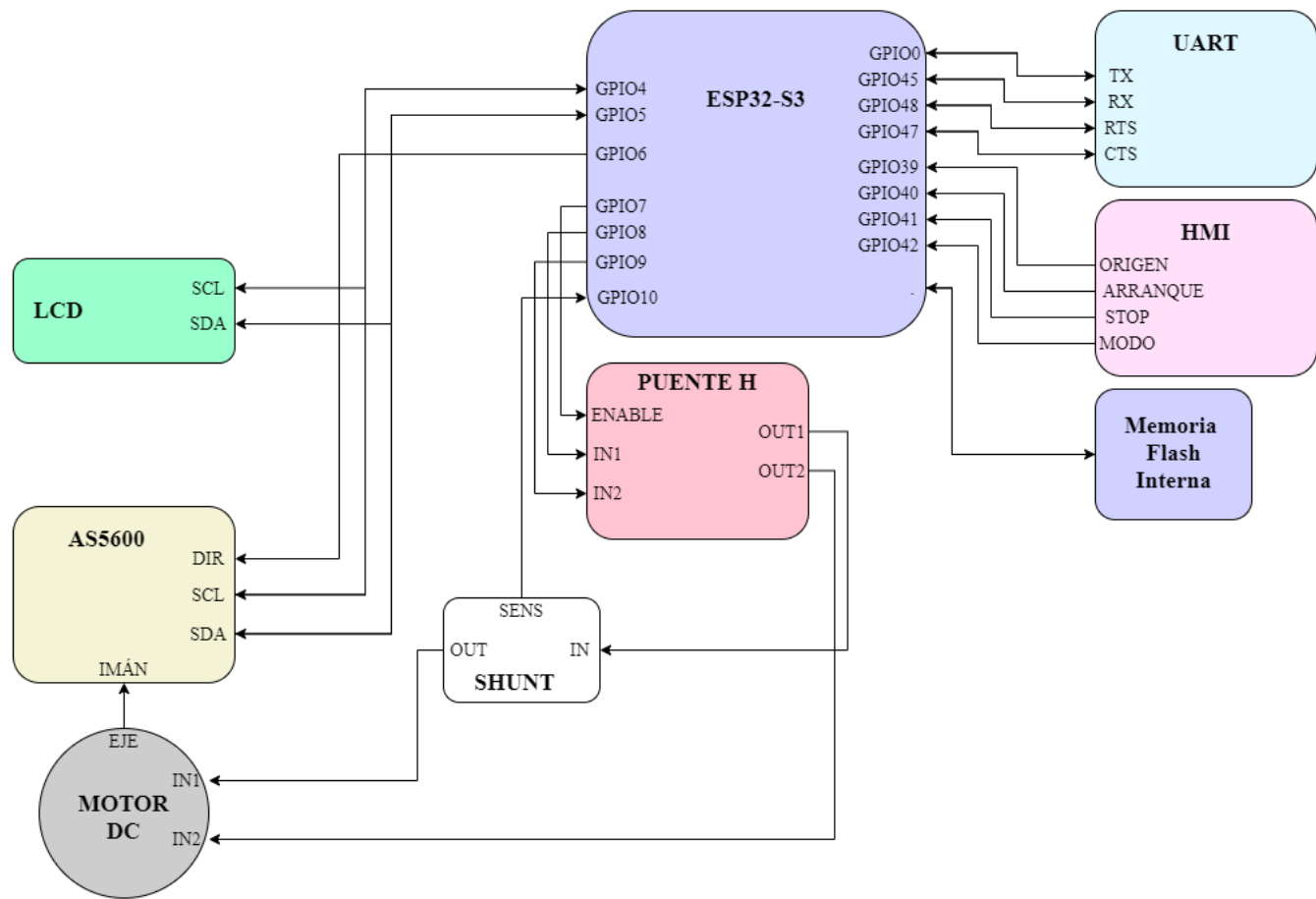


Fig. 14 Diagrama de tareas del controlador PID.

Anexo 2. Diagrama de tareas

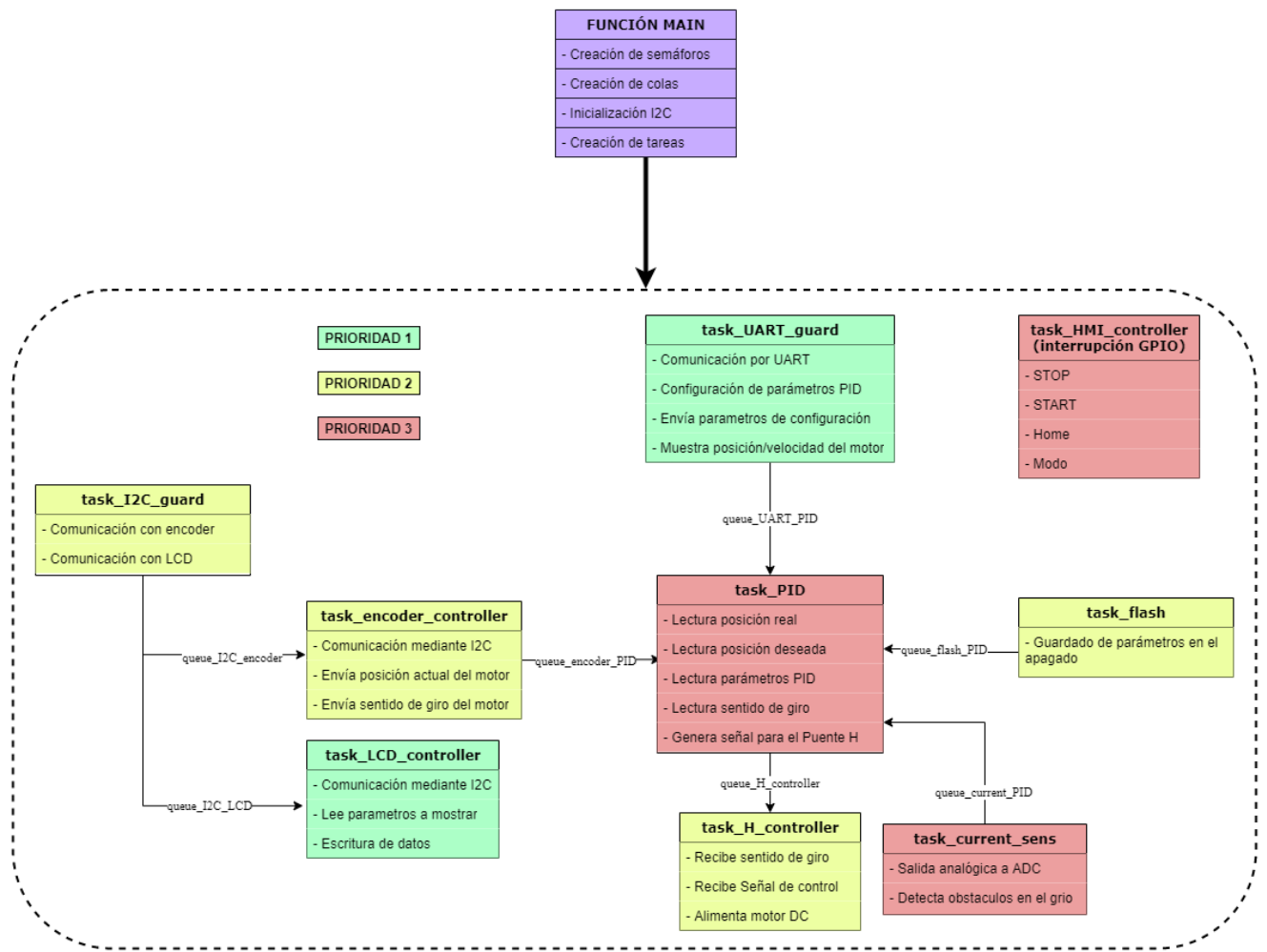


Fig. 15 Diagrama de tareas del controlador PID.